Rapport technique du projet Fhemni

De la collecte des données à l'évaluation du système RAG

1. Présentation générale du projet

Fhemni est un projet qui vise à simplifier la compréhension des courriers administratifs et juridiques en expliquant leur contenu en darija marocaine, tout en s'appuyant sur des sources juridiques officielles.

L'objectif est de permettre à un utilisateur de comprendre :

- Le sujet principal d'un courrier.
- Les montants financiers mentionnés.
- Les délais à respecter.
- Les démarches à effectuer.
- Les références juridiques qui justifient l'explication.

Pour réaliser cet objectif, nous avons utilisé une architecture RAG (*Retrieval-Augmented Generation*), qui combine la recherche de documents pertinents avec un modèle de langage, Apertus.

2. Organisation des fichiers et des dossiers

Le projet est organisé en plusieurs parties qui correspondent aux différentes étapes du traitement des données, de la recherche documentaire et de la génération des réponses.

2.1. Le dossier `data/sources/`

Ce dossier contient les documents juridiques utilisés comme base de connaissances pour le système.

Il comprend 22 fichiers de textes juridiques officiels provenant de Fedlex, organisés selon trois catégories :

- `assurance_maladie/` : documents relatifs à l'assurance maladie.
- `rappel_poursuite/` : documents relatifs aux rappels de paiement, aux poursuites et aux procédures de recouvrement des dettes.
- `sejour/` : documents relatifs au séjour et aux autorisations de séjour.

Pourquoi cette organisation ?

Elle permet de classer les textes par domaine juridique et de faciliter leur identification et leur maintenance. Ces documents constituent la source de référence utilisée par le système pour retrouver les informations nécessaires à l'explication des courriers.

2.2. Le fichier `src/ingest.py`

Ce fichier est responsable de la phase d'ingestion des données (*Data Ingestion*).

Il automatise la préparation des documents pour qu'ils puissent être exploités par le système de recherche.

Son rôle dans le pipeline est le suivant :

1. Récupérer les documents juridiques disponibles dans les sources.
2. Lire et extraire leur contenu textuel.
3. Diviser les textes en petits segments appelés *chunks*.
4. Générer les représentations vectorielles (*embeddings*) à l'aide de BGE-M3.
5. Enregistrer les segments et leurs informations associées dans Chroma.

La séquence exacte dépend de l'implémentation du fichier et de ses fonctions. Le principe général est de préparer les documents pour permettre une recherche sémantique ultérieure.

Point important concernant mon travail : je n'ai pas nécessairement découpé manuellement chaque document. Le travail consiste à préparer et exécuter le processus d'ingestion à travers `ingest.py`, qui automatise les opérations prévues dans le code.

Le résultat enregistré lors de la première préparation était de 67 chunks.

2.3. Les embeddings avec BGE-M3

BGE-M3 est le modèle utilisé pour représenter les segments de texte sous forme de vecteurs numériques.

Un embedding représente les caractéristiques sémantiques d'un texte. Il permet de rechercher des passages proches du sens d'une question, même si celle-ci ne reprend pas exactement les mots du document original.

Par exemple, un utilisateur peut demander en darija ce qu'il doit faire après avoir reçu un rappel de paiement. Le système cherche alors les passages juridiques correspondant à cette situation.

L'objectif n'est donc pas seulement de chercher des mots identiques, mais de retrouver des informations pertinentes par leur sens.

2.4. Chroma : stockage des données vectorielles

Chroma est la base de données vectorielle utilisée pour conserver les chunks et leurs embeddings, ainsi que les métadonnées associées lorsqu'elles sont enregistrées.

Son rôle est de permettre au système de retrouver les segments juridiques les plus pertinents lorsqu'un utilisateur soumet un courrier ou une question.

Il faut distinguer les rôles des composants :

- BGE-M3 transforme les textes en vecteurs.
- Chroma stocke les vecteurs et permet de les rechercher.

- Le module de retrieval exploite cette recherche pour récupérer les passages utiles.

2.5. Le fichier `src/retrieve.py`

Ce fichier prend en charge la phase de récupération des informations (*Retrieval*).

Lorsqu'un utilisateur fournit un courrier, le système recherche dans Chroma les chunks susceptibles de contenir les informations nécessaires.

Les résultats sont ensuite transmis à la partie du système chargée de générer l'explication.

Par exemple, si le courrier concerne une autorisation de séjour, le système doit privilégier les documents du domaine `sejour/`, plutôt que des passages portant uniquement sur l'assurance maladie.

Le rôle de cette étape est essentiel : même un bon modèle de langage peut produire une réponse incorrecte si les documents récupérés ne correspondent pas à la situation.

2.6. Les fichiers de génération et de pipeline

Les fichiers suivants font également partie du travail global du projet, d'après l'organisation décrite par l'équipe :

- `llm.py` : associé à l'interaction avec le modèle de langage.
- Les prompts : définissent les consignes données au modèle, notamment pour expliquer les courriers en darija et structurer les réponses.
- Le pipeline : coordonne les différentes étapes, de la réception du courrier jusqu'à la production de la réponse.
- `check.py` : associé aux vérifications du système.
- L'application Gradio : fournit une interface permettant à l'utilisateur d'interagir avec le système.
- Docker : permet de préparer un environnement d'exécution reproductible.
- Le glossaire et les exemples en darija native : contribuent à améliorer la qualité linguistique et la compréhension des réponses.

Les chemins exacts de tous ces éléments doivent être confirmés dans l'arborescence réelle du dépôt. Il ne faut pas supposer que chacun se trouve directement dans `src/`.

3. Fonctionnement global du système RAG

Le système suit le processus suivant :

Sources juridiques → Ingestion → Chunking → Embeddings → Chroma → Retrieval → Apertus → Réponse en darija

Le fonctionnement se déroule en deux phases.

Phase A : préparation des documents

Cette phase est réalisée avant les requêtes des utilisateurs.

1. Les documents juridiques sont collectés.
2. `ingest.py` prépare les textes et les découpe en segments.
3. BGE-M3 génère les embeddings.
4. Les données sont enregistrées dans Chroma.

Phase B : traitement d'un courrier

Cette phase est déclenchée lorsqu'un utilisateur soumet un courrier.

1. Le système reçoit le texte du courrier.
2. `retrieve.py` recherche les passages juridiques pertinents.
3. Le pipeline transmet le courrier et le contexte récupéré au modèle Apertus.
4. Le modèle produit une explication destinée à l'utilisateur.
5. Le système peut structurer la réponse et effectuer des vérifications selon son implémentation.

Le principe fondamental est que le modèle ne doit pas se baser uniquement sur ses connaissances générales : il doit aussi exploiter le contexte juridique récupéré.

4. Benchmark : comparaison entre Apertus 8B et Apertus 70B

Nous avons effectué des tests préliminaires avec deux versions du modèle Apertus : 8B et 70B.

Le suffixe B signifie *billion*, c'est-à-dire milliard de paramètres. Le modèle 8B possède environ 8 milliards de paramètres, contre environ 70 milliards pour le modèle 70B.

Un modèle plus grand peut avoir davantage de capacité, mais cela ne garantit pas qu'il sera plus performant sur une tâche particulière.

Résultats préliminaires

Critère	Apertus 8B	Apertus 70B
Scores obtenus lors des tests préliminaires	Identiques	Identiques
Vitesse d'exécution	Plus rapide	Environ 1,7 fois plus lent
Taille du modèle	Plus petite	Plus grande
Coût de calcul potentiel	Généralement inférieur	Généralement supérieur

Ces observations correspondent aux résultats préliminaires enregistrés pour notre projet. Elles ne constituent pas une garantie que les deux modèles auront exactement la même qualité sur toutes les tâches.

Pourquoi peut-on préférer Apertus 8B ?

Le choix d'Apertus 8B se justifie principalement par **l'efficacité et le compromis entre qualité et temps d'exécution**.

Premièrement, les deux modèles ont obtenu les mêmes scores dans les tests préliminaires réalisés. Le modèle 70B n'a donc pas démontré d'amélioration mesurable sur ce benchmark.

Deuxièmement, Apertus 70B était environ 1,7 fois plus lent. Le modèle 8B permet ainsi d'obtenir les mêmes résultats observés en utilisant moins de temps pour la génération.

Troisièmement, un modèle plus petit demande généralement moins de mémoire et de ressources de calcul, ce qui peut faciliter le déploiement. La consommation réelle dépend toutefois de la quantification, du matériel et des paramètres d'inférence.

Conclusion du benchmark : dans les conditions de nos tests préliminaires, Apertus 8B représente le choix le plus efficace, car il fournit les mêmes scores observés avec une génération plus rapide.

Il serait incorrect d'affirmer qu'Apertus 8B est intrinsèquement plus intelligent ou plus précis qu'Apertus 70B. Nous pouvons seulement affirmer qu'il a présenté un meilleur compromis dans les expériences effectuées.

5. Évaluation du système

Pour vérifier le fonctionnement de Fhemni, nous avons préparé un jeu de test comprenant 20 courriers administratifs synthétiques, avec des informations attendues et des références juridiques à retrouver.

Les tests ont porté sur plusieurs aspects du système.

Indicateur	Résultat préliminaire	Interprétation
JSON valide	100 %	Les sorties respectaient le format JSON attendu dans les tests.
Extraction des montants	95 %	Les montants ont été correctement extraits dans la grande majorité des cas.
Extraction des délais	85 %	Certains délais n'ont pas été correctement identifiés.
Retrieval Hit@3	81 %	La référence attendue apparaissait parmi les trois premiers résultats dans 81 % des cas évalués.

Ces résultats sont préliminaires et doivent être confirmés après les dernières corrections.

Interprétation des résultats

Le taux de JSON valide montre que le système sait produire une sortie structurée dans le format demandé. Cependant, un JSON valide ne signifie pas nécessairement que son contenu est juridiquement correct.

Le taux d'extraction des montants indique que cette partie du traitement fonctionne relativement bien sur le jeu de test utilisé.

L'extraction des délais est plus délicate, car un même courrier peut contenir plusieurs délais correspondant à des démarches différentes.

Enfin, Hit@3 mesure la qualité de la récupération documentaire. Il ne mesure ni la précision complète de la réponse générée, ni la conformité juridique de toutes les explications.

Problème identifié : confusion entre les délais

Un problème observé concerne la distinction entre :

- Le délai de paiement.
- Le délai d'opposition à une poursuite.

Ces deux délais correspondent à des démarches différentes et ne doivent pas être confondus.

Cette observation montre que le système doit améliorer l'identification du contexte juridique, la récupération des références exactes et la génération des explications.

6. Répartition du travail dans l'équipe

Ma contribution

Ma contribution principale concerne la préparation des données, la recherche documentaire et l'évaluation du système.

Elle comprend notamment :

1. La collecte et l'organisation des textes juridiques.
2. La préparation du processus d'ingestion avec `ingest.py`.
3. La mise en place de la récupération documentaire avec `retrieve.py`.
4. La préparation de 20 courriers synthétiques pour les tests.
5. L'élaboration de scripts d'évaluation et l'analyse des résultats.
6. La comparaison préliminaire d'Apertus 8B et d'Apertus 70B.

Il est important de présenter cette contribution en fonction du code réellement réalisé et des tests effectivement exécutés.

Contribution de mon coéquipier

La contribution de mon coéquipier porte principalement sur :

- L'intégration du modèle de langage et du pipeline.
- L'amélioration des prompts.
- Les vérifications et la structuration des réponses.
- L'interface utilisateur avec Gradio.
- La préparation de Docker et du README.

- Le glossaire et les exemples de darija native.

Cette répartition permet de distinguer la partie consacrée à la préparation et à l'évaluation des données de celle consacrée à l'interaction avec le modèle et à l'expérience utilisateur.

7. Travaux restant à effectuer

Plusieurs améliorations restent nécessaires avant de considérer les résultats comme définitifs.

- Appliquer et vérifier la version corrigée de `ingest.py`.
- Réexécuter l'ingestion et confirmer le nombre final de chunks.
- Relancer les tests d'évaluation.
- Tester le système sur de nouveaux courriers, différents de ceux utilisés pendant le développement.
- Améliorer la distinction entre les délais juridiques.
- Vérifier les références légales citées dans les réponses.
- Faire évaluer la qualité de la darija par des locuteurs natifs.
- Finaliser le README, Docker et la présentation du projet.

8. Conclusion générale

Fhemni repose sur une architecture RAG qui combine des documents juridiques officiels, des embeddings BGE-M3, une base vectorielle Chroma et un modèle de langage Apertus.

Le travail réalisé sur la préparation des données et la récupération documentaire constitue une étape fondamentale : la pertinence des sources récupérées influence directement la qualité des explications produites.

Les tests préliminaires ont montré des résultats encourageants, notamment pour la validation du JSON et l'extraction des montants. Ils ont également révélé des difficultés dans l'extraction des délais et dans la sélection des références juridiques.

Enfin, la comparaison entre Apertus 8B et Apertus 70B suggère que le modèle 8B est le choix le plus efficace dans les conditions testées, puisque les scores observés étaient identiques alors que le modèle 70B était environ 1,7 fois plus lent.

L'objectif final est de proposer un système capable d'expliquer les courriers administratifs en darija marocaine de manière claire, compréhensible et fondée sur des références juridiques pertinentes.